T.C.

FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

E-MAIL SYSTEMS

PROJECT 1

Name and Surname: Sema Nur ÖZGEN

Number: 2421021011

Examining the class structures that make up the email system:

base classes:

1. Dashboard.java:

    - This class includes the “Dashboard” module of an email management system. Users can view their inbox, send emails and access personalization options. The database uses a MySQL database called mailsystem to store emails.
    - Provides interaction with the e-mail system through the user interface (UI).

    Display the inbox (show_inboxActionPerformed method).

    Identifying the user and assigning their information to the UI component.

    Switch to a new email sending screen.

    Switch to user customization (personalize) screen.

    - Table: mail

    Fields

    id: Unique ID.

    subject: Email subject.

    sender: Sender username.

    data_received: Date received.

    message: Message content.

    recipient: Recipient username.

    category: Email category (e.g. 'Sent').

    - Main Tab:

    JTabbedPane to display inbox information.

    JLabel for user name (user_name).

    - Components:

    table_inbox: A JTable listing the inbox.

    Columns: id, subject, sender, data_received, message.

2. LoginScreen.java

- Overview:The provided LoginScreen class implements a GUI-based login interface for an email system using Java Swing. It interacts with a MySQL database to authenticate users and includes functionalities like showing/hiding passwords, signing up, and using Google accounts (currently under development).

- Functionalities:

Login with Credentials: Verifies user email and password against the MySQL database.
Sign Up Option: Redirects to a register form for new user registration.
Show Password: Toggles visibility of the password field.
Google Login: Displays a placeholder message for future development.
Database Integration:
Uses MySQL for storing and retrieving user information.
Verifies email and password using SQL queries.
Error Handling:
Validates user inputs (email and password).
Provides user-friendly error messages for empty fields and incorrect credentials.
Object-Oriented Structure:
Encapsulation of user details via the Email class.
Modularized event handling with methods for each component action.

3. RoundButton Class

- The RoundButton class extends JButton to create a custom round button for a Java Swing GUI application. This custom button is visually distinct due to its rounded appearance and specific styling. Below is a detailed analysis of the class.

4. Feedback Class: Code Analysis and Report

- The feedback class is a Java Swing-based graphical user interface (GUI) component that allows users to submit feedback as either a suggestion or a complaint. The feedback is stored in a database. This class is part of a larger application, as suggested by its integration with other components and database interactions.Two JRadioButton components allow users to specify the feedback type (suggestion or complaint).

5. Personalize.java

- The personalize class provides an interface that allows the user to set their personal preferences in the system. This interface allows users to manage their favorite email addresses, change the screen theme, choose the maximum send size and set other personal preferences.

6. register.java

- The Register class provides a registration screen that allows users to create a new account. It takes full name, email and password information from the user, validates this information and after validation saves the information in a MySQL database. It also provides the possibility to switch from the registration screen to the login screen.

7. sendEmail.java

- This class contains a Java Swing interface that allows a user to send e-mail. The following main functions are included:

Retrieving the sender address (sender1) from the logged in user.
Retrieving the recipient address (txt_to), subject (txt_subject) and message content (txt_message) fields from the user.
Validation of the fields (no blanks, banned word check, etc.).
Saving the message in the MySQL database.

Helper Classes: DBConnector , MailSender Class and UserSession

1. MailSender Class

- The MailSender class is designed to perform basic operations related to email. It includes functions such as sending emails, updating email status and listing emails by a specific category.

2.UserSession Class

- The UserSession class is designed to manage and store the information of the logged in user throughout the application. When a user logs in or out, this class is used to retain or reset user-specific information such as email address.

2. DBConnector Class

- The DBConnector class is a helper class designed to facilitate connection to the MySQL database used in the project. It provides reusability by keeping the database connection details in a central place.

The places where I had difficulty:

- One of the difficulties I had in UserSession was dynamically passing the logged in user's e-mail address to other classes. For example, in the sendEmail class, I wanted to retrieve the logged in user's email address from the UserSession.loggedInUserEmail variable. However, I noticed that in some cases this variable returned null, suggesting that the logged in user's information was not logged correctly or was not retrieved correctly when called from other classes. To solve this problem, I had to check the login and logout processes more carefully.

- I had a few difficulties in the Database Connection (DBConnector and General SQL Operations) section. First of all, I encountered some connection errors when connecting to the database. For example, although I made sure that the MySQL connection URL, username or password were correct, I sometimes experienced situations such as not being able to establish a connection. Also, when writing SQL queries, I encountered mismatches with data types or missing parameter errors. For example, when inserting data into the database using a PreparedStatement, in some cases I mistyped the order of the parameters or realized that the data types did not match (for example, expecting int instead of String). This caused the query to stop working and I encountered SQLException errors. In addition, when adding or updating data to the database, I may have forgotten to catch situations such as some required fields being blank. In such cases, I had to add extra validations so that the error messages were more descriptive and I could detect problems faster. In the

process, I learned to check my SQL queries carefully and make sure that I bind each parameter correctly.